Data Structures & Systems Design Assignment

Shaurya Bajpai | AI / Multi-Agent Systems Intern, Avaloka (Guruvai Sciences) |
bajpaishaurya2911@gmail.com | +91-7985200306

Everything is in Python 3. I ran both solutions against my own test cases before writing this up.

Problem 1: LRU Cache

How I thought about it. A cache like this has to do two things quickly: find a key, and know which key I have not touched for the longest time. A dictionary finds keys in O(1) but has no idea about order. A linked list can hold the order, but searching it is slow. So I used both.

The dictionary maps each key to a node, and the nodes sit in a doubly linked list. The front of the list is the most recently used item and the back is the least recently used. The list has to be doubly linked because to pull a node out from the middle I need to reach its previous node too, and the dictionary already hands me the node directly, so no searching is needed.

In **get**, I look the key up, move its node to the front, and return the value (or -1 if it is missing). In **put**, if the key already exists I update it and move it to the front. If it is new and the cache is full, I drop the node at the back, delete its key from the dictionary, and then add the new node at the front. That is why each node keeps its own key: I need it to clean up the dictionary on eviction. I also used two dummy nodes at both ends so I never have to check for None.

```
class Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=0, value=0):
        self.key, self.value = key, value
        self.prev = self.next = None


class LRUCache:
    def __init__(self, capacity):
        if capacity <= 0:
            raise ValueError("capacity must be positive")
        self.capacity = capacity
        self.map = {}
        # sentinels: head.next is most recent, tail.prev is least recent
        self.head, self.tail = Node(), Node()
        self.head.next, self.tail.prev = self.tail, self.head

    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev

    def _add_front(self, node):
        node.prev, node.next = self.head, self.head.next
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        node = self.map.get(key)
        if node is None:
            return -1
        self._remove(node)
        self._add_front(node)
        return node.value

    def put(self, key, value):
        node = self.map.get(key)
        if node:
            node.value = value
            self._remove(node)
            self._add_front(node)
            return
        if len(self.map) == self.capacity:
            lru = self.tail.prev
```

```
            self._remove(lru)
            del self.map[lru.key]
        node = Node(key, value)
        self.map[key] = node
        self._add_front(node)
```

Problem 2: Event Scheduler

can_attend_all. I sort the events by start time and then look at each neighbouring pair. If one event ends after the next one starts, there is a clash. I compare with *end* <= *next start*, so (9,10) followed by (10,11) is fine, as the problem says.

min_rooms_required. The number of rooms I need is the most meetings happening at the same time, so I just track that. I sort all the start times in one list and all the end times in another. Then I go through the starts one by one. Before I count a new meeting, I free up every room whose meeting has already ended (end time <= this start). Then I add the new meeting and update the highest count seen so far. That highest count is the answer. Using <= when freeing rooms is what lets back-to-back meetings share one room.

```
def can_attend_all(events):
    ev = sorted(events)
    return all(ev[i][1] <= ev[i + 1][0] for i in range(len(ev) - 1))


def min_rooms_required(events):
    if not events:
        return 0
    starts = sorted(s for s, _ in events)
    ends = sorted(e for _, e in events)
    rooms = best = j = 0
    for s in starts:
        while ends[j] <= s:  # a meeting ending at s frees its room first
            j += 1
            rooms -= 1
        rooms += 1
        best = max(best, rooms)
    return best
```

Final Discussion & Analysis

1. Time and space complexity (n is the number of events)

Function	Time	Space
LRUCache.get	O(1)	O(1)
LRUCache.put	O(1)	O(1)
LRU cache overall	n/a	O(capacity)
can_attend_all	O(n log n), the sort dominates	O(n) for the sorted copy
min_rooms_required	O(n log n), two sorts; the sweep itself is O(n)	O(n)
assign_rooms (see 3)	O(n log n)	O(n)

2. Why a hash map and a doubly linked list? Because each one alone fails at something. With only a hash map, I can find a key fast but I cannot tell which one is the oldest without scanning everything, which is O(n). With only a list or array, I keep the order but finding a key is O(n), and shifting items around in an array is O(n) as well. A singly linked list is also not enough, since removing a node needs its previous node. Using both, the map jumps straight to the node and the list lets me move or delete it in O(1). The price is some extra memory, one map entry and two pointers per item, which I think is worth it. (The OrderedDict in Python is basically this same idea, but I wanted to build it myself to show how it works.)

3. Assigning room names. For this I would use two min-heaps: one for rooms that are busy, ordered by when they finish, and one for room names that are free. I go through events sorted by start time. At each event I move every busy room whose meeting has finished into the free heap. If there is a free room I use it,

otherwise I open a new one (Room A, then Room B, and so on). Then I push the room onto the busy heap with the end time of the event. The total rooms opened ends up equal to min_rooms_required, and the time is O(n log n). I keep the original index of each event so the answer can be matched back to the input order. On [(0,30),(5,10),(15,20)] it gives Room A, Room B, Room B.

```
import heapq

def assign_rooms(events):
    """Returns {event_index: 'Room A'...}; reuses the earliest-free room."""
    order = sorted(range(len(events)), key=lambda i: events[i])
    busy, free, out, n = [], [], {}, 0  # busy: (end, room_id); free: heap of ids
    for i in order:
        s, e = events[i]
        while busy and busy[0][0] <= s:
            heapq.heappush(free, heapq.heappop(busy)[1])
        if free:
            r = heapq.heappop(free)
        else:
            r, n = n, n + 1
        out[i] = "Room " + chr(ord("A") + r) if r < 26 else f"Room {r + 1}"
        heapq.heappush(busy, (e, r))
    return out
```

4. Making the LRU Cache thread-safe. The tricky part is that even *get* changes the list, since it moves a node to the front. So I cannot treat reads as harmless and use a read-write lock; every call needs exclusive access. The simple fix is a single threading.Lock (or RLock) wrapped around get and put with *with self.lock:*. This keeps the dictionary and the list in sync. The downside is that threads wait on each other, so under heavy load it becomes a bottleneck. If that mattered, I would split the cache into several smaller caches by hash(key) % N, each with its own lock, accepting that eviction is then only approximately LRU. Another option is to batch the recency updates instead of doing them on every read, which is what libraries like Caffeine do. In Python the GIL protects single operations, but get and put have several steps, so the lock is still needed.